

[Introduction](#)

[Set up the environment](#)

[Understand the monitoring script](#)

[Run the monitoring script](#)

[View monitoring data in Azure Monitor](#)

[View trace data](#)

[\(OPTIONAL\). Create an alert](#)

[\(OPTIONAL\). Add a fourth prompt version](#)

[Where to find other labs](#)

Monitor and trace your generative AI agent

This exercise takes approximately **40 minutes**.

! **Note:** This lab assumes a pre-configured lab environment with Visual Studio Code, Azure CLI, and Python already installed.

Introduction

In this exercise, you'll use Application Insights and distributed tracing to observe the Trail Guide Agent's runtime behavior. After iterating on prompts in earlier labs, you now have a concrete question: what did those prompt changes actually cost in the real world?

Scenario: You're operating the Adventure Works Trail Guide Agent and have deployed three prompt versions over time. Before deciding which version to keep in production, you want to observe how each one performs at runtime — measuring token usage, latency, and response behavior across a consistent set of test inputs.

You'll run the same five test prompts against prompt versions v1, v2, and v3, then analyze the results from two angles:

- **Azure Monitor:** Aggregated token usage and latency metrics across versions
- **Trace tree (local):** A nested span tree per version from `python src/tests/check_traces.py`, with per-prompt timing and token attributes

Set up the environment

To complete the tasks in this exercise, you need:

- Visual Studio Code
- Azure subscription with Microsoft Foundry access
- Git and [GitHub](#) account
- Python 3.9 or later
- Azure CLI and Azure Developer CLI (azd) installed

! **Tip:** If you haven't installed these prerequisites yet, see [Lab 00: Prerequisites](#) for installation instructions and links.

All steps in this lab will be performed using Visual Studio Code and its integrated terminal.

Create repository from template

You'll start by creating your own repository from the template to practice realistic workflows.

1. In a web browser, navigate to the template repository on [GitHub](#) at `https://github.com/MicrosoftLearning/mslearn-genaiops`.
2. Select **Use this template** > **Create a new repository**.
3. Enter a name for your repository (e.g., `mslearn-genaiops`).
4. Set the repository to **Public** or **Private** based on your preference.
5. Select **Create repository**.

Clone the repository in Visual Studio Code

After creating your repository, clone it to your local machine.

1. In Visual Studio Code, open the Command Palette by pressing **Ctrl+Shift+P**.
2. Type **Git: Clone** and select it.
3. Enter your repository URL: `https://github.com/[your-username]/mslearn-genaiops.git`
4. Select a location on your local machine to clone the repository.
5. When prompted, select **Open** to open the cloned repository in VS Code.

Deploy Microsoft Foundry resources

Now you'll use the Azure Developer CLI to deploy all required Azure resources.

1. In Visual Studio Code, open a terminal by selecting **Terminal** > **New Terminal** from the menu.
2. Authenticate with Azure Developer CLI:

Code

Copy

```
azd auth login
```

This opens a browser window for Azure authentication. Sign in with your Azure credentials.

3. Authenticate with Azure CLI:

Code

Copy

```
az login
```

Sign in with your Azure credentials when prompted.

!

⚠

Important

In some environments, the VS Code integrated terminal may crash or close during the interactive login flow. If this happens, authenticate using explicit credentials instead:

Code

Copy

```
az login --username <your-username> --password <your-password>
```

4. Provision resources:

Code

Copy

```
azd up
```

!

Note:

If `azd up` fails because the default model deployment is unavailable in your region, update the `aiProjectDeploymentsJson` block in `infra/main.bicep` to a compatible model and rerun the command.

When prompted, provide:

- **Environment name** (e.g., `dev` , `test`) - Used to name all resources
- **Azure subscription** - Where resources will be created
- **Location** - Azure region (recommended: Sweden Central)

The command deploys the infrastructure from the `infra\` folder, creating:

- **Resource Group** - Container for all resources
- **Foundry (AI Services)** - The hub with access to models like GPT-5.1
- **Foundry Project** - Your workspace for creating and managing agents
- **Log Analytics Workspace** - Collects logs and telemetry data
- **Application Insights** - Monitors performance and usage

5. Create a `.env` file with the environment variables:

Code

Copy

```
azd env get-values > .env
```

This creates a `.env` file in your project root with all the provisioned resource information.

!

⚠

Important

– File

Encoding

After generating the `.env` file, make sure it is saved using **UTF-8** encoding.

In editors like **VS Code**, check the encoding indicator in the bottom-right corner.

If it shows **UTF-16 LE** (or any encoding other than UTF-8), click it, choose **Save with Encoding**, and select **UTF-8**.

Using the wrong encoding may cause environment variables to be read incorrectly.

This creates a `.env` file in your project root with all the provisioned resource information.

Install Python dependencies

With your Azure resources deployed, install the required Python packages.

1. In the VS Code terminal, create and activate a virtual environment:

CodeCopy

```
python -m venv .venv
.venv\Scripts\Activate.ps1
```

2. Install the required dependencies:

CodeCopy

```
python -m pip install -r requirements.txt
```

This installs all required packages, including:

- `azure-monitor-opentelemetry` - Exports telemetry to Application Insights
- `opentelemetry-instrumentation-openai-v2` - Auto-instruments chat completion calls

3. Add the model configuration to your `.env` file.

Open the `.env` file in your repository root and add:

CodeCopy

```
MODEL_NAME="<model_name>"
```

!

Note: Set `MODEL_NAME` to the deployed model you want to trace in your environment. For example, if you kept the template default and it is available in your region, you can use `gpt-5.1`.

Understand the monitoring script

Before running anything, take a moment to review what `src/tests/run_monitoring.py` does.

1. Open `src/tests/run_monitoring.py` in Visual Studio Code.
2. Review the **telemetry setup** at the top of the script:

CodeCopy

```
connection_string = project_client.telemetry.get_application_insights_connection_string()
configure_azure_monitor(connection_string=connection_string)
OpenAIInstrumentor().instrument()
```

This retrieves the Application Insights connection string that was provisioned alongside your Foundry project. `OpenAIInstrumentor` automatically creates child spans for every `chat.completions.create()` call without any extra code.

3. Review the `run_version` **function**. It wraps each prompt version in a top-level span named `trail_guide_v1`, `trail_guide_v2`, or `trail_guide_v3`, and wraps each individual test inside a child span. Custom attributes like `response.total_tokens` and `response.duration_s` are attached to every child span.
4. Note the `VERSIONS` **list** at the top:

Code

```
VERSIONS = ["v1", "v2", "v3"]
```

The script loads each version’s system prompt from `src/agents/trail_guide_agent/prompts/` and runs the same five test prompts from `src/tests/test-prompts/` against it. This gives you a consistent, reproducible comparison.

Run the monitoring script

1. In the VS Code terminal, run the script:

Code

```
python src/tests/run_monitoring.py
```

2. Watch the terminal output as the script progresses through each version. For each test prompt you’ll see:
- **Duration:** How long the model took to respond
 - **Tokens:** Total tokens used, broken down into prompt and completion tokens
 - **Response:** A preview of the model’s output
3. Note any visible differences between versions as they run. For example:
- Does v3 produce noticeably longer responses than v1?
 - Does token count grow consistently from v1 to v3?
4. View the trace tree for the latest run (recommended).

This lab uses a local trace viewer script that queries Log Analytics and prints a nested tree. This makes the parent/child chain very explicit without relying on a portal UI.

Run:

Code

```
python src/tests/check_traces.py
```

You should see one trace per prompt version, with three levels:

Code

```
trail_guide_v1                                ← root: entire version run
├── v1_day-hike-gear                          ← child: one test prompt (duration + token
attributes)
|   ├── chat gpt-5.1                          ← auto: the actual LLM call
└── ...
trail_guide_v2  (same structure)
trail_guide_v3  (same structure)
```

If the output says `No traces found yet`, wait 2–3 minutes and run the script again.

View monitoring data in Azure Monitor

Now you’ll examine the aggregated performance metrics for all three versions.

Navigate to Azure Monitor from Microsoft Foundry

1. In a web browser, open the [Microsoft Foundry portal](#) at `https://ai.azure.com` and sign in using your Azure credentials.
2. Open your Foundry project.
3. In the navigation pane on the left, select **Monitoring**.
4. Select **Resource usage** to see a summary of model interactions.

 **Note:** If no data appears yet, wait a minute and refresh the page.

Review token usage across versions

Focus on the **token usage** metrics and compare the three prompt versions:

- **Total token count:** The combined total of prompt and completion tokens across all calls.

 This is the primary cost driver. A jump between versions here signals that the prompt change has a direct financial impact.

- **Prompt token count:** Tokens used in the input (the system prompt plus the user message).

 Longer, more detailed system prompts like v3 will show higher prompt token counts even for identical user questions.

- **Completion token count:** Tokens the model generated in its response.

 If v3 produces significantly more completion tokens than v1, the model is generating more verbose responses — which increases both cost and latency.

- **Total requests:** The total number of model calls. You should see 15 requests — five test prompts across three versions.

Compare the individual interactions

Use the output of `python src/tests/check_traces.py` to compare prompt versions.

1. For the same test name (for example, `v1_trail-difficulty` vs `v3_trail-difficulty`), compare:
 - **Duration** (shown in milliseconds)
 - **Total tokens** (and prompt vs completion breakdown)
2. Use the nested structure to understand the chain:
 - The `v{n}_{test-name}` span is where you attach per-test token and duration attributes.
 - The `chat gpt-5.1` span is the instrumented model call.
 - Which version produces the most consistent response lengths?
 - Which version shows the highest duration?
 - Are there any calls that stand out as unusually slow or verbose?

View trace data

The `check_traces.py` output gives you the full span tree for each version run — one root span per version, with nested child spans for each test prompt and auto-instrumented LLM calls beneath those.

Review the version spans

1. Run the trace viewer script:

Code

 Copy

```
python src/tests/check_traces.py
```

2. Locate the root spans: `trail_guide_v1` , `trail_guide_v2` , and `trail_guide_v3` .
3. Under each root span, locate a test child span like `v1_trail-difficulty` .
4. Review the inline attributes printed on each test child span:

Attribute	What it tells you
Span name (for example, <code>v3_trail-difficulty</code>)	Which prompt version + which test prompt
<code>[...]ms</code>	Latency for that span (milliseconds)
<code>tokens: total (↑prompt ↓completion)</code>	Token usage for that specific test call

Compare span trees across versions

1. Compare `trail_guide_v3` to `trail_guide_v1` for the same test (for example, `v3_trail-difficulty` vs `v1_trail-difficulty`).
 - Are completion tokens higher in v3?
 - Is the duration longer?
 - Compare the `chat gpt-5.1` child span durations under each test span.
2. Repeat the comparison for at least two other test prompts. Document your observations:
 - Which version shows the best balance of response quality and token efficiency?
 - Does the increase in prompt complexity (v1 → v3) produce proportionally better responses, or is the extra cost hard to justify?

(OPTIONAL) Create an alert

If you have extra time, set up an alert to notify you when token usage exceeds a threshold. This is an exercise designed to challenge you, which means instructions are intentionally less detailed.

- In Azure Monitor, create a **new alert rule** for your Microsoft Foundry project.
- Choose a metric such as **Total token count** and define a threshold that would be exceeded if you ran the script again (based on what you observed above).
- Create a **new action group** to define how you'll be notified.

Alerts help you proactively catch unexpected spikes in usage before they translate into budget overruns.

(OPTIONAL) Add a fourth prompt version

If you completed [Lab 03: Design and optimize prompts](#), you already evaluated `v4_optimized_concise` and found it reduced token usage significantly. Now verify that at runtime:

1. Open `src/tests/run_monitoring.py` and update the `VERSIONS` list:

Code

```
VERSIONS = ["v1", "v2", "v3", "v4_optimized_concise"]
```

Copy

2. Run the script again:

Code

```
python src/tests/run_monitoring.py
```

Copy

3. Run `python src/tests/check_traces.py` again and compare the new `trail_guide_v4_optimized_concise` span tree against `trail_guide_v3` .
 - Does the token reduction you measured in evaluation hold up at runtime?

- Is there a latency improvement as well?
- Does the quality of responses in the spans appear comparable?

Where to find other labs

You can explore additional labs and exercises in the [Microsoft Foundry portal](#) or refer to the course's **lab section** for other available activities.